

Exercise: Poultry

Martin Johnsson

Part 1: Quantitative genetic parameters

In this first example, we use body weight in broiler chickens as an illustration of quantitative genetic parameters. The following code blocks generate the figures and numbers that were used in the lecture.

Run through them and inspect the results.

```
library(AlphaSimR)
library(dplyr)
library(ggplot2)
library(tibble)
library(tidyr)
```

First, like many times before, we generate founder haplotypes with `runMacs` and the “GENERIC” population history.

```
founders <- runMacs(nInd = 1000,
                   nChr = 10)
```

We set up a simulation without changing any of the default settings, and add a trait that includes dominance. The mean and phenotypic variance and the heritability come from Dadousis et al. (2021), whereas the distribution for the dominance degree is unknown, so we use hypothetical parameters.

```
simparam <- SimParam$new(founders)
simparam$addTraitAD(nQtlPerChr = 1000,
                   mean = 1840,
                   var = 191^2,
                   meanDD = 0,
                   varDD = 0.1)
simparam$setVarE(h2 = 0.44)
```

We generate the population, and take the 50 first animals for plotting purposes.

```
pop_full <- newPop(founders,  
                  simParam = simparam)  
  
pop <- pop_full[1:50]
```

Genetic values

Here is a data frame of quantitative genetic values calculated from different slots of the population object and summary functions. Look into the calculation of each column, look at documentation to the functions and understand what they do.

```
stats <- tibble(id = pop@id,  
              pheno = pop@pheno[,1],  
              P = pheno - meanP(pop),  
              gv = gv(pop)[,1],  
              G = gv - meanP(pop),  
              E = P - G,  
              A = bv(pop, simParam = simparam)[,1],  
              D = dd(pop, simParam = simparam)[,1])
```

The 50 animals are sorted for plotting purposes.

```
stats <- stats[order(stats$pheno, decreasing = FALSE),]  
stats$sorted_id <- 1:nrow(stats)
```

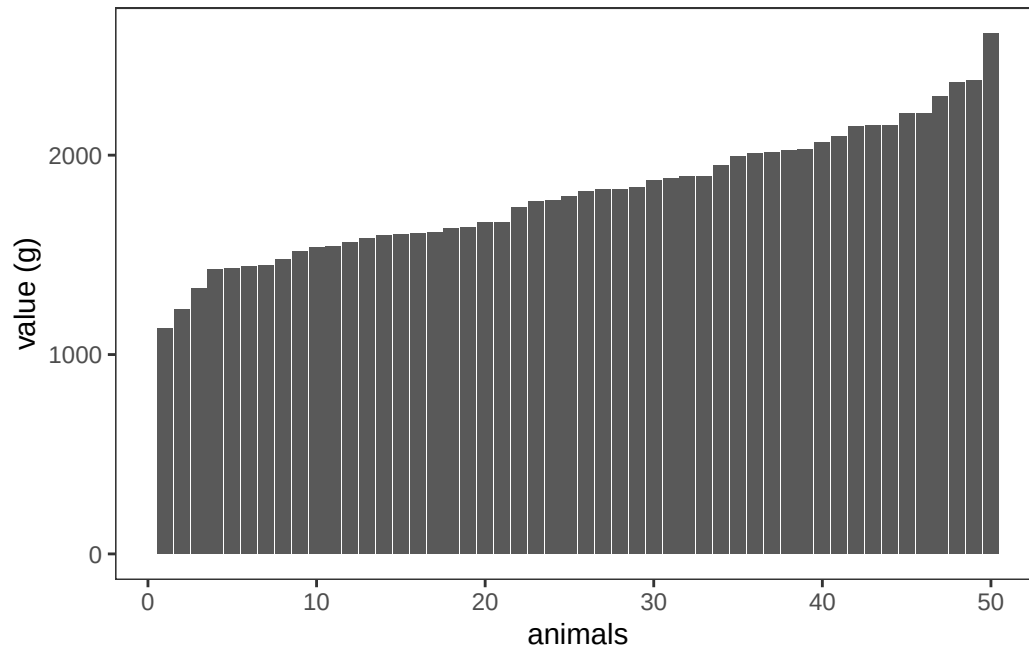
A function from the `tidyr` packages i used to pivot the data into a different format for plotting.

```
stats_long <- pivot_longer(stats, -c(id, sorted_id))
```

First plot of phenotypes:

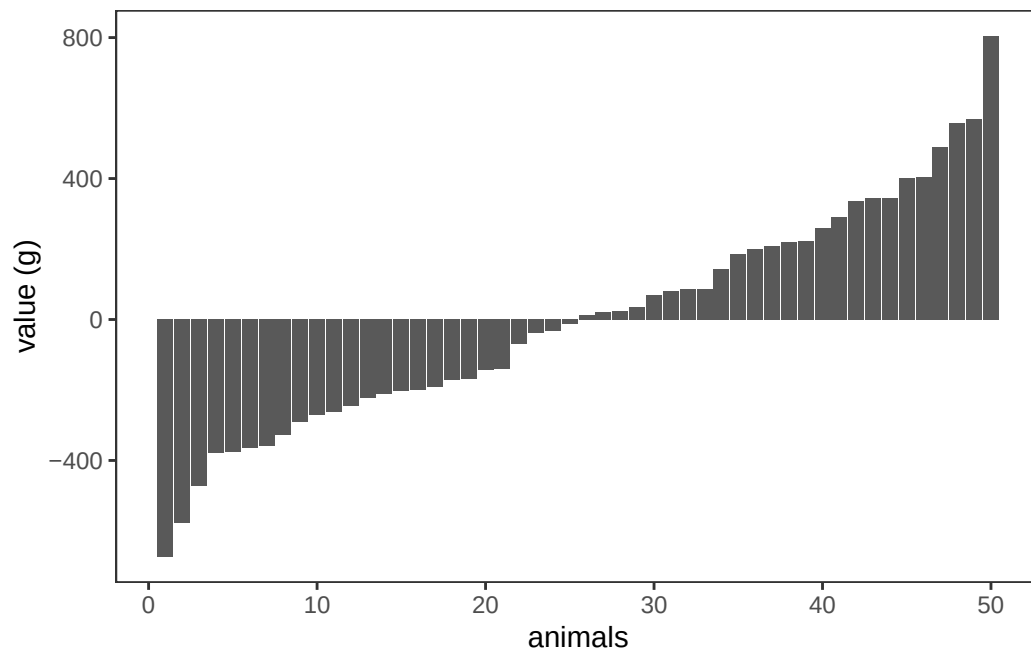
```
formatting <- list(theme_bw(),  
                  theme(panel.grid = element_blank(),  
                        legend.title = element_blank()),  
                  xlab("animals"),  
                  ylab("value (g)"))
```

```
plot_pheno <- ggplot() +
  geom_bar(aes(x = sorted_id,
               y = pheno),
           stat = "identity",
           data = stats) +
  formatting
plot_pheno
```



Plot of phenotype expressed as deviation from population mean.

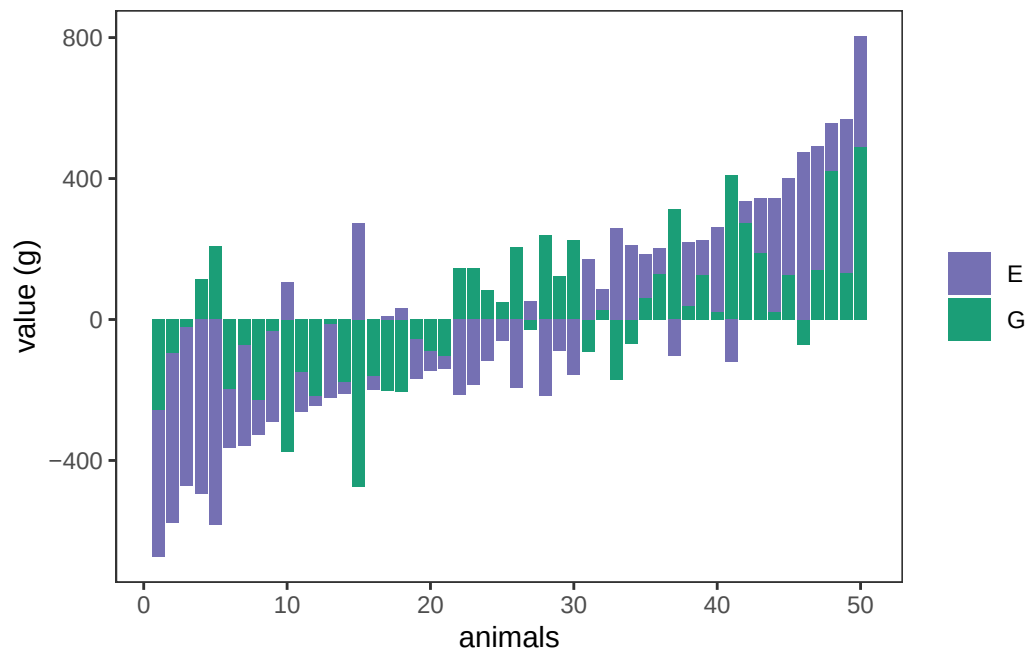
```
plot_pheno_corrected <- ggplot() +
  geom_bar(aes(x = sorted_id,
               y = P),
           stat = "identity",
           data = stats) +
  formatting
plot_pheno_corrected
```



Plot of basic decomposition in genetic and environmental values.

```
plot_GE <- ggplot() +
  geom_bar(aes(x = sorted_id,
              y = value,
              fill = name),
          stat = "identity",
          position = position_stack(),
          data = filter(stats_long,
                        name %in% c("G", "E")))) +
  formatting +
  scale_fill_manual(values = c("#7570b3", "#1b9e77"))

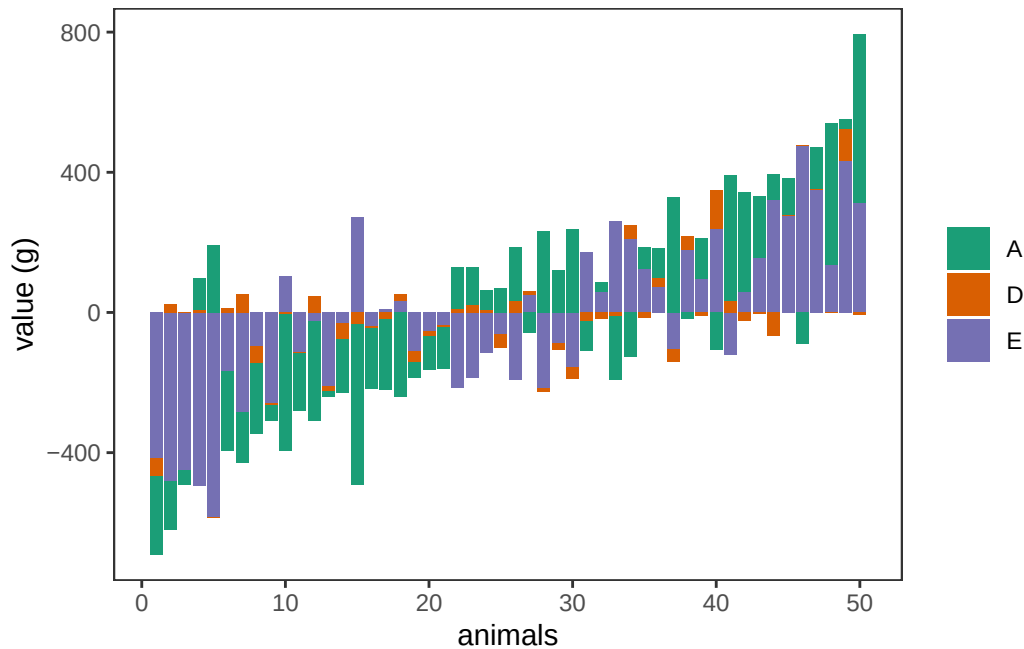
plot_GE
```



Plot of finer decomposition into additive genetic, dominance deviation and environmental values.

```
plot_ADE <- ggplot() +
  geom_bar(aes(x = sorted_id,
               y = value,
               fill = name),
           stat = "identity",
           position = position_stack(),
           data = filter(stats_long,
                        name %in% c("A", "D", "E"))) +
  formatting +
  scale_fill_manual(values = c("#1b9e77", "#d95f02", "#7570b3"))

plot_ADE
```



Response to selection

This block simulates one round of phenotypic selection and mating, generating 100 offspring, and measures the observed response compared to selection response predicted from the breeders equation. Look through the calculation and compare to the breeders equation.

```
selected <- selectInd(pop = pop_full,
                     nInd = 100,
                     simParam = simparam)

mean_selected <- meanP(selected)
mean_all <- meanP(pop_full)

selection_diff <- mean_selected - mean_all

pred_response <- selection_diff * 0.44

offspring <- randCross(pop = selected,
                      nCrosses = 1000,
                      simParam = simparam)
```

```
mean_offspring <- meanP(offspring)

selection_response <- mean_offspring - mean_all

selection_response
```

```
Trait1
210.9894
```

```
pred_response
```

```
Trait1
221.0059
```

Finally, we illustrate the selection with histograms of the population, selected parents, and offspring.

```
hist_data <- tibble(population = c(rep("population", pop_full@nInd),
                                   rep("selected", selected@nInd),
                                   rep("offspring", offspring@nInd)),
                   pheno = c(pop_full@pheno[,1],
                              selected@pheno[,1],
                              offspring@pheno[,1]))

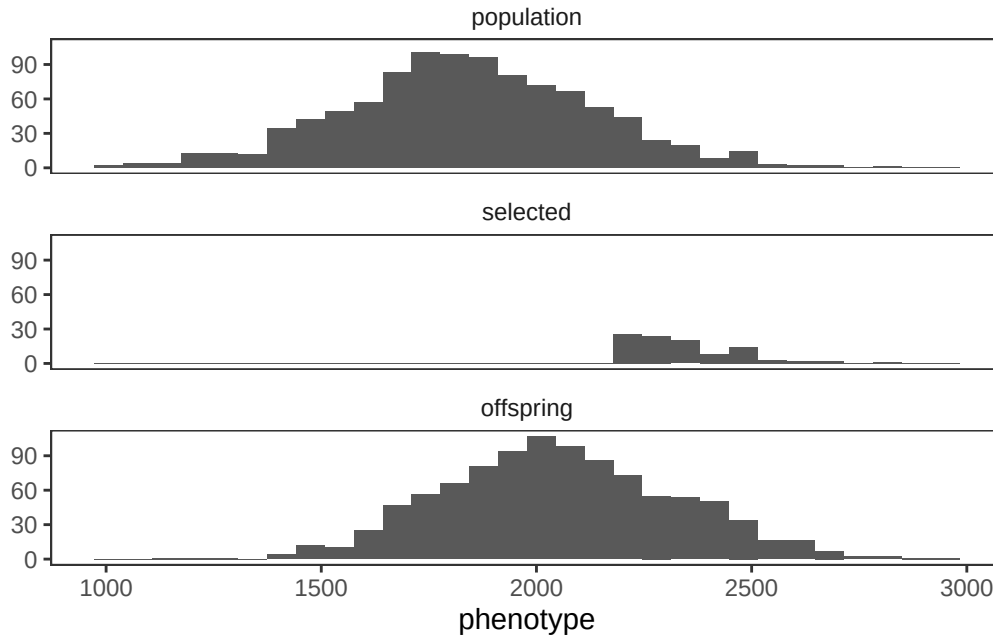
hist_data$population <- factor(hist_data$population,
                               levels = c("population", "selected", "offspring"))

plot_histograms <- qplot(x = pheno, data = hist_data) +
  facet_wrap(~population, ncol = 1) +
  theme_bw() +
  theme(panel.grid = element_blank(),
        strip.background = element_blank()) +
  xlab("phenotype")
```

Warning: `qplot()` was deprecated in ggplot2 3.4.0.

```
plot_histograms
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Exercises

Select the exercises that interest you.

- **Replication of selection response.** Use loops or functions to perform the simulation of selection, calculation of observed selection response and prediction of the selection response from the breeders equation multiple times. Summarise the results. How well does the simulation and the breeders equation agree?
- **Dominance.** Change the mean and/or variance of the dominance degree (in `simparam$addTraitAD`) and see how it affects the response to selection.
- **Epistasis.** Look into the `addTraitADE` function (e.g., `simparam$addTraitADE`) for adding a trait with both additive, dominance and epistatic locus-level coefficients. Make a plot similar to the plot of additive genetic, dominance deviation and environmental values that also includes the epistatic interaction deviation.

Part 2: The preselection model of de Hollander et al. (2024)

The below code implements part of the simulation of de Hollander et al. (2024) of preselection in a broiler chicken breeding program.

As usual, we start by setting up a founder population and settings (sexes on).

de Hollander et al. (2024) use different simulation tools and therefore set up their populations in a different way. Look at their methods section and compare.

```
founders <- runMacs(nInd = 1200,  
                  nChr = 10)  
  
simparam <- SimParam$new(founders)  
  
simparam$setSexes("yes_sys")
```

We add three correlated traits. See de Hollander et al. (2024) for details.

```
trait_correlation_matrix <- matrix(c(1, 0.11, 0.15,  
                                     0.11, 1, 0.14,  
                                     0.15, 0.14, 1),  
                                   ncol = 3,  
                                   byrow = TRUE)  
  
simparam$addTraitA(nQtlPerChr = 100,  
                  mean = c(0, 0, 0),  
                  var = c(1, 1, 1),  
                  corA = trait_correlation_matrix)  
  
simparam$setVarE(h2 = c(0.18, 0.19, 0.17))
```

We create a founder population and split it by sex to be able to set up the starting populations.

```
founder_population <- newPop(founders,  
                             simParam = simparam)  
  
males <- selectInd(founder_population,  
                  nInd = 10 * 4,  
                  sex = "M",  
                  use = "rand",  
                  simParam = simparam)  
females <- selectInd(founder_population,  
                    nInd = 150 * 4,  
                    sex = "F",  
                    use = "rand",  
                    simParam = simparam)
```

We population all stages of the breeding program with males and females.

```

sires <- list(males[1:10],
             males[11:20],
             males[21:30],
             males[31:40])

dams <- list(females[1:150],
            females[151:300],
            females[301:450],
            females[451:600])

```

Bookkeeping. We set up variables for storing genetic means of all traits, separately for males and females, accuracy of preselection, and variables that indicate the number of progeny in each group and the number of time steps we will run.

```

n_steps <- 40

n_progeny_group <- c(16, 16, 15, 13)

sire_genetic_mean_t1 <- numeric(n_steps)
sire_genetic_mean_t2 <- numeric(n_steps)
sire_genetic_mean_t3 <- numeric(n_steps)
dam_genetic_mean_t1 <- numeric(n_steps)
dam_genetic_mean_t2 <- numeric(n_steps)
dam_genetic_mean_t3 <- numeric(n_steps)

accuracy_preselection <- numeric(n_steps)

```

Main breeding loop

At each time step, we first record genetic means, and then run “stage 0” (generating offspring, “stage 1” (preselection), and stages 3-4 (maturation and selection on index), and the final stages where we advance the breeders and keep them for four hatches (see de Hollander et al. 2024 for details).

```

for (step in 1:n_steps) {
  sire_means <- meanG(mergePops(sires))
  dam_means <- meanG(mergePops(dams))
  sire_genetic_mean_t1[step] <- sire_means[1]
  sire_genetic_mean_t2[step] <- sire_means[2]
  sire_genetic_mean_t3[step] <- sire_means[3]

```

```

dam_genetic_mean_t1[step] <- dam_means[1]
dam_genetic_mean_t2[step] <- dam_means[2]
dam_genetic_mean_t3[step] <- dam_means[3]

## "Stage 0": offspring created from the 4 breeder groups
chicks <- vector(length = 4, mode = "list")

for (group in 1:4) {
  chicks[[group]] <- randCross2(
    males = sires[[group]],
    females = dams[[group]],
    nCrosses = dams[[group]]@nInd,
    nProgeny = n_progeny_group[group],
    simParam = simparam)
}

offspring <- mergePops(chicks)

## "Stage 1": preselection based on body weight (trait 1)
males_preselected <- selectInd(offspring,
  sex = "M",
  nInd = 4500 * 0.1,
  trait = 1,
  use = "pheno",
  simParam = simparam)

females_preselected <- selectInd(offspring,
  sex = "F",
  nInd = 4500 * 0.3,
  trait = 1,
  use = "pheno",
  simParam = simparam)

accuracy_preselection[step] <- cor(offspring@gv[,1], offspring@pheno[,1])

## "Stage 4": selection based on index (of true breeding values)
males_selected <- selectInd(males_preselected,
  nInd = 10,
  trait = selIndex,
  b = c(1, 0.82, 0.5),
  use = "gv",
  simParam = simparam)

```

```

females_selected <- selectInd(females_preselected,
                              nInd = 150,
                              trait = selIndex,
                              b = c(1, 0.82, 0.5),
                              use = "gv",
                              simParam = simparam)

## Advance stages 5-8
sires <- list(males_selected, sires[[1]], sires[[2]], sires[[3]])
dams <- list(females_selected, dams[[1]], dams[[2]], dams[[3]])
}

```

Results

We collect the results in a data frame and pivot for easier plotting.

```

results <- data.frame(step = 1:n_steps,
                      sire_genetic_mean_t1,
                      sire_genetic_mean_t2,
                      sire_genetic_mean_t3,
                      dam_genetic_mean_t1,
                      dam_genetic_mean_t2,
                      dam_genetic_mean_t3,
                      accuracy_preselection)

results_long <- pivot_longer(results, -c(step, accuracy_preselection))

```

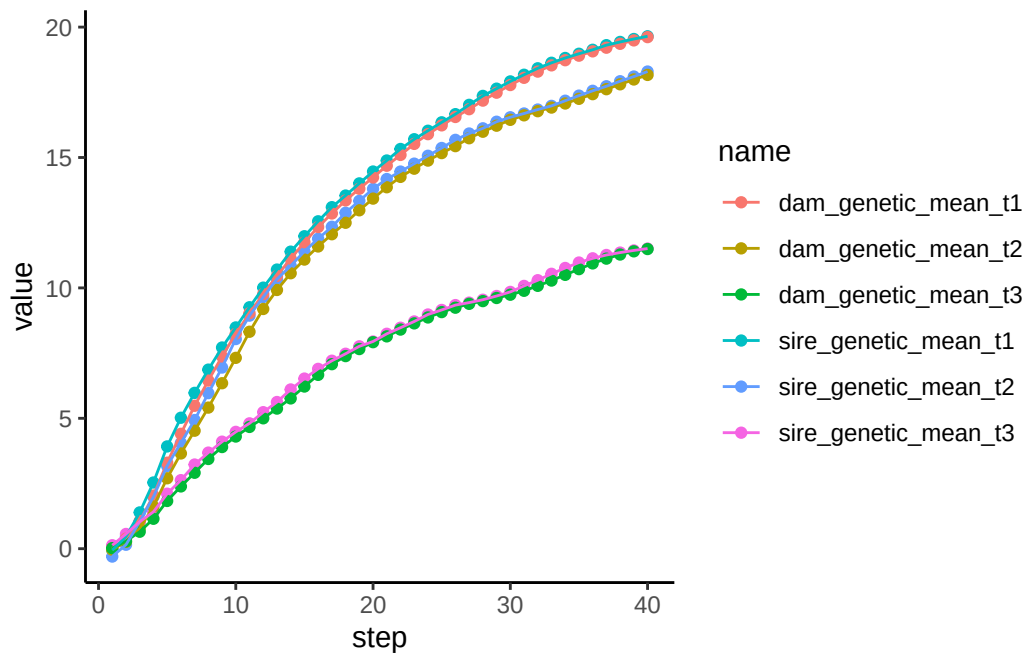
Plot of genetic means over time steps.

```

plot_means <- ggplot(aes(x = step, y = value, colour = name),
                     data = results_long) +
  geom_point() + geom_line() +
  theme_classic()

plot_means

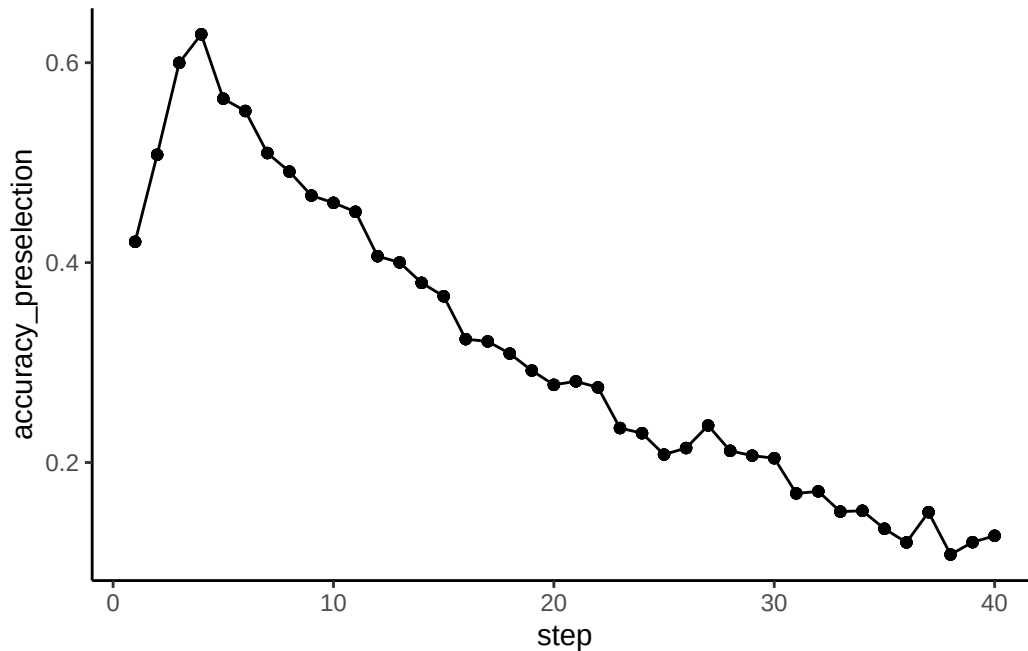
```



Plot of accuracy of pre-selection.

```
plot_accuracy <- ggplot(aes(x = step, y = accuracy_preselection),
                        data = results_long) +
  geom_point() + geom_line() +
  theme_classic()

plot_accuracy
```



Exercises

Choose the exercises that interest you.

- **Varying preselection strategy.** The above code only implements one of the preselection intensities in the paper (10% of males, 30% of females). Implement one or both of the other options and compare them with respect to genetic gain. You should probably also replicate the simulation to make sure that any differences you see are not due to random variation.
- **Genomic evaluation.** The above code performs preselection index selection on true genetic values, which would correspond to perfectly accurate selection. That is obviously unrealistic. Use the `RRBLUP` function to implement genomic prediction so that you can make the final selection on GEBV instead. In order to do this you need to: 1) add a SNP chip `simparam$addSnpChip()`, 2) set up a training population made out of older animals with records, 3) run GBLUP with the `RRBLUP` function, 4) use the `setEBV` function to set EBVs of the selection candidates, and finally 5) change the `use="gv"` argument to `use="ebv"`. Because genomic evaluation takes some time to run, you might want to keep the training population small and only run a few time steps with genomic evaluation.
- **Reading.** Read part of de Hollander et al. (2024) and consider the aims of their simulation. What key features are missing from the above code to be able to address their research questions? What would need to be implemented?